

Day 1: Cloud Training & Git Bash Basics

Introduction to Shell Environments, Commands, and Server Architecture

Module 1: Git Bash & Command Line Interface (CLI)

Git Bash is a shell utility for Windows environments that provides emulation of the Unix command line. It acts as a Bourne Again SHell (Bash) wrapper, giving Windows users access to standard Unix utilities, paths, and command syntax. In Cloud Computing and DevOps pipelines, Linux is the standard operating system. Git Bash allows Windows-based developers to practice local navigation and terminal operations identical to remote cloud server configurations.

Basic Navigational Concepts

- **Paths Structure:** Windows filesystems use backslashes (e.g. C:\Users\Jeeva), whereas Git Bash maps system partitions using forward slashes (e.g. /c/Users/Jeeva) mirroring Unix conventions.
- **Case Sensitivity:** Commands and filenames are strictly case-sensitive. Input commands must be executed in precise lowercase characters.

Essential Terminal Commands Reference

Command	Example	Detailed Description
pwd	pwd	Print Working Directory. Outputs the absolute path of the directory currently active.
ls	ls OR ls -la	List Files. Prints names of items in the folder. '-la' flags show details and hidden files.
mkdir	mkdir new_folder	Make Directory. Creates a new subfolder with the specified name in current path.
cd	cd /c/devops	Change Directory. Moves focus to the folder path provided in the argument.
cd ..	cd ..	Parent Directory. Navigates one level higher into the parent directory folder.
touch	touch app.js	Create File. Quickly instantiates an empty text file or script file.
rm	rm target.txt	Remove. Deletes a file. 'rm -rf' removes folders and all children recursively.
clear	clear	Refresh. Resets the terminal screen buffer to remove previous outputs.

Module 2: Server Infrastructure (Physical vs. Virtual)

Modern application deployment relies on raw bare-metal hardware and abstraction virtualization layers. DevOps engineers must understand how host systems partition physical compute hardware to optimize workloads.

What is a Physical Server?

A physical server (commonly called a Bare-Metal server) is a dedicated computer system. It contains

physical components including high-performance motherboards, enterprise CPUs (Intel Xeon, AMD EPYC), ECC RAM modules, physical storage disks (SATA/NVMe SSDs), power supplies (PSUs), and network interfaces (NICs). The operating system is installed directly onto the hardware without virtualization layers, making it a single-tenant environment.

- **Advantages:** Maximum performance, direct access to raw compute power, no interference from shared resource neighbors, full hardware config control.
- **Disadvantages:** High procurement costs, slow scaling (requires physical installations), maintenance overhead, low average utilization (usually 10-15%).

What is a Virtual Server?

A virtual server (or Virtual Machine - VM) is a software-based computer running inside a physical server host. A specialized software layer called a Hypervisor partition the physical server's CPU, memory, storage, and networking resources into isolated blocks, presenting them to separate Guest Operating Systems. This allows multi-tenancy where multiple independent virtual servers run on a single hardware host.

- **Advantages:** Extremely cost-effective, rapid provisioning (starts in seconds), easy recovery with backups/snapshots, high portability between hosts.
- **Disadvantages:** Slight translation overhead from the Hypervisor layer, risk of resource contention if neighboring VMs consume shared resources.